SIMATS ENGINEERING ASSESSMENT
TOOL 1

NAME: LAHARI REDDY **REG NO: 192411164**
COURSE NAME: INTERNET PROGRAMMING **COURSE CODE: CSA43**

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool: Application-Based Questions

Weightage: 40%

QUESTIONS: **Total Marks: 88**

1: Online Symposium Portal

A college is developing an online symposium registration portal. The following HTML structure and HTTP interaction is observed:

```
<form action="/register" method="POST">
  <input type="text" name="name">
  <input type="email" name="email">
  <button>Submit</button>
</form>
```

When a student submits the form, the browser sends an HTTP request to the server, and the server responds accordingly. **Questions:**

1. Identify appropriate **HTML5 semantic elements** missing in the structure.
2. Modify the structure to make it **standards-compliant**.
3. Interpret the **HTTP request generated** during form submission.
4. Analyze the **HTTP response cycle** from server to browser.
5. Suggest improvements for **usability and accessibility**.

2: Cross-Browser Compatibility Issue

A company website shows inconsistent layout across browsers. The following HTML snippet is used:

```
<html>
  <head>
    <title>Company</title>
  </head>
  <body>
    <div>
      <h1>Welcome
      <p>About us
    </div>
  </body> </html>
```

Questions:

1. Identify **improper nesting and missing closing tags**.
2. Analyze how different browsers may interpret this structure.
3. Identify **missing semantic elements** (e.g., header, section).
4. Provide a **corrected W3C-compliant HTML structure**.
5. Suggest techniques to ensure **cross-browser compatibility**.

3: Form Submission Failure

A web application fails to send form data to the server. The following configuration is observed:

```
<form action="/submit" method="GET">
  <input type="text" name="username">
  <input type="password" name="password">
  <button type="button">Login</button>
</form>
```

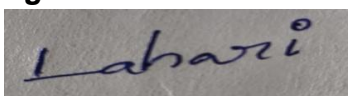
Questions:

1. Identify issues in the **form configuration**.
2. Analyze why the **HTTP request is not triggered**.
3. Interpret the role of **GET method in this scenario**.
4. Propose a **corrected implementation**.
5. Suggest improvements for **secure data transmission**.

Code of Conduct:

I LAHARI REDDY *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weig htage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	MARKS
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconception s	Poor understandin g; major misconceptio ns	**22**
Explanation	25%	Detailed, wellexplained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation	**23**
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis	**17**
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers	**13**
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps	**9**
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand	**4**

1. Online Symposium Portal

Q1. Identify appropriate HTML5 semantic elements missing in the structure.

- <header> – Defines the page heading.
- <main> – Contains the main content.
- <section> – Groups related registration content.
- <footer> – Defines footer information.
- <label> – Provides labels for input fields.

Q2. Modify the structure to make it standards-compliant.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Symposium Registration</title>
</head>
<body>
  <header>
    <h1>Online Symposium Portal</h1>
  </header>
  <main>
    <section>
      <h2>Student Registration</h2>
      <form action="/register" method="POST">
        <label for="name">Name:</label>
        <input type="text" id="name" name="name" required>

        <label for="email">Email:</label>
        <input type="email" id="email" name="email" required>

        <button type="submit">Submit</button>
      </form>
    </section>
  </main>
  <footer>
    <p>College Symposium 2026</p>
  </footer>
</body>
```

</html>

Q3. Interpret the HTTP request generated during form submission.

When the student clicks Submit, the browser sends an HTTP POST request to the /register endpoint.

Example:

```
POST /register HTTP/1.1
Host: college.edu
Content-Type: application/x-www-form-urlencoded

name=Ravi&email=ravi%40example.com
```

- POST: Sends data in the request body.
- Host: Identifies the server.
- Content-Type: Specifies the data format.
- Request body: Contains the student's name and email.

Q4. Analyze the HTTP response cycle from server to browser.

1. The student submits the registration form.
2. The browser sends the HTTP request to the server.
3. The server validates and processes the data.
4. The server sends an HTTP response with a status code and response content.
5. The browser displays a success message or an error.

Examples of response codes: 200 OK, 201 Created, 400 Bad Request, and 500 Internal Server Error.

Q5. Suggest improvements for usability and accessibility.

1. Add labels to all input fields.
2. Use required for mandatory fields.
3. Use appropriate input types, such as email.
4. Display clear success and error messages.
5. Support keyboard navigation and screen readers.
6. Use HTTPS to protect data during transmission.

2. Cross-Browser Compatibility Issue

Q1. Identify improper nesting and missing closing tags.

The given HTML has the following problems:

- The <h1> element is missing its closing </h1> tag.
- The element is missing its closing tag.
- The document is missing the <!DOCTYPE html> declaration.
- The <html> element is missing the lang attribute.
- Character encoding and viewport metadata are missing.

Q2. Analyze how different browsers may interpret this structure.

Different browsers may automatically correct malformed HTML using their parsing rules. This may result in differences in the DOM structure, spacing, layout, and CSS rendering. Therefore, properly written HTML is necessary for consistent behavior.

Q3. Identify missing semantic elements.

- <header> – Contains the page heading.
- <main> – Contains the main content.
- <section> – Groups the company information.
- <footer> – Contains copyright or contact information.

Q4. Provide a corrected W3C-compliant HTML structure.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Company</title>
</head>
<body>
  <header>
    <h1>Welcome</h1>
  </header>
  <main>
    <section>
      <h2>About Us</h2>
      <p>Learn more about our company.</p>
    </section>
  </main>
  <footer>
    <p>Copyright 2026 Company</p>
  </footer>
</body>
</html>
```

Q5. Suggest techniques to ensure cross-browser compatibility.

1. Use valid HTML5 and CSS.
2. Include the <!DOCTYPE html> declaration.
3. Use CSS resets or consistent baseline styles.
4. Implement responsive design using media queries.
5. Test the website in Chrome, Firefox, Edge, and Safari.
6. Validate the code using the W3C HTML Validator.

3. Form Submission Failure

Q1. Identify issues in the form configuration.

- The button uses type="button" instead of type="submit".
- The GET method is inappropriate for sending login passwords.
- Input fields lack labels.
- Required-field validation is missing.
- Server-side authentication and validation are necessary.

Q2. Analyze why the HTTP request is not triggered.

The button has type="button", which does not automatically submit the form. Therefore, clicking Login does not normally send the form data to the server. Changing it to type="submit" triggers the default form submission behavior.

Q3. Interpret the role of the GET method in this scenario.

The GET method appends form data to the URL as query parameters.
Example:
GET /submit?username=ravi&password=abc123 HTTP/1.1
Problems:

- Passwords may appear in browser history and server logs.
- URLs can be copied or shared accidentally.
- GET is primarily intended for retrieving resources.

Therefore, POST over HTTPS is preferred for login forms. HTTPS encrypts data during transmission.

Q4. Propose a corrected implementation.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Login</title>
</head>
<body>
  <main>
    <h1>Student Login</h1>
    <form action="/submit" method="POST">
      <label for="username">Username:</label>
      <input type="text" id="username"
          name="username" required>

      <label for="password">Password:</label>
      <input type="password" id="password"
```

```
            name="password" required>

        <button type="submit">Login</button>
      </form>
    </main>
</body>
</html>
```

1. Use HTTPS with a valid TLS certificate.
2. Use POST instead of GET for login credentials.
3. Validate user input on both the client and server.
4. Store passwords using secure hashing algorithms such as Argon2id or bcrypt.
5. Use secure session cookies.
6. Apply rate limiting to prevent brute-force attacks.
7. Never rely only on client-side validation for security.